

Visualisation & Web App

Applications web

Xavier Gendre 

Application web

Une application web (ou *web app*) est une application utilisable via un navigateur (*Firefox, Chrome, ...*).

En particulier, une telle application est installée sur un **serveur** et ne nécessite pas d'installer des programmes spécifiques sur la machine **cliente**.

Exemples classiques :

- Moteurs de recherche
- Messagerie
- Wikis
- Forum
- Jeux en ligne
- ...

Un *framework web* est généralement utilisé pour développer une application web. Il s'agit d'un ensemble d'outils logiciels permettant de réaliser les opérations courantes (accès aux bases de données, ...) et d'automatiser le développement web.

Flask et django sont des frameworks web Python connus.



Objectif

Mettre en place des applications web pour la visualisation de données.

Ces applications sont souvent appelées des **tableaux de bord interactifs** (ou *interactive (data) dashboard*).

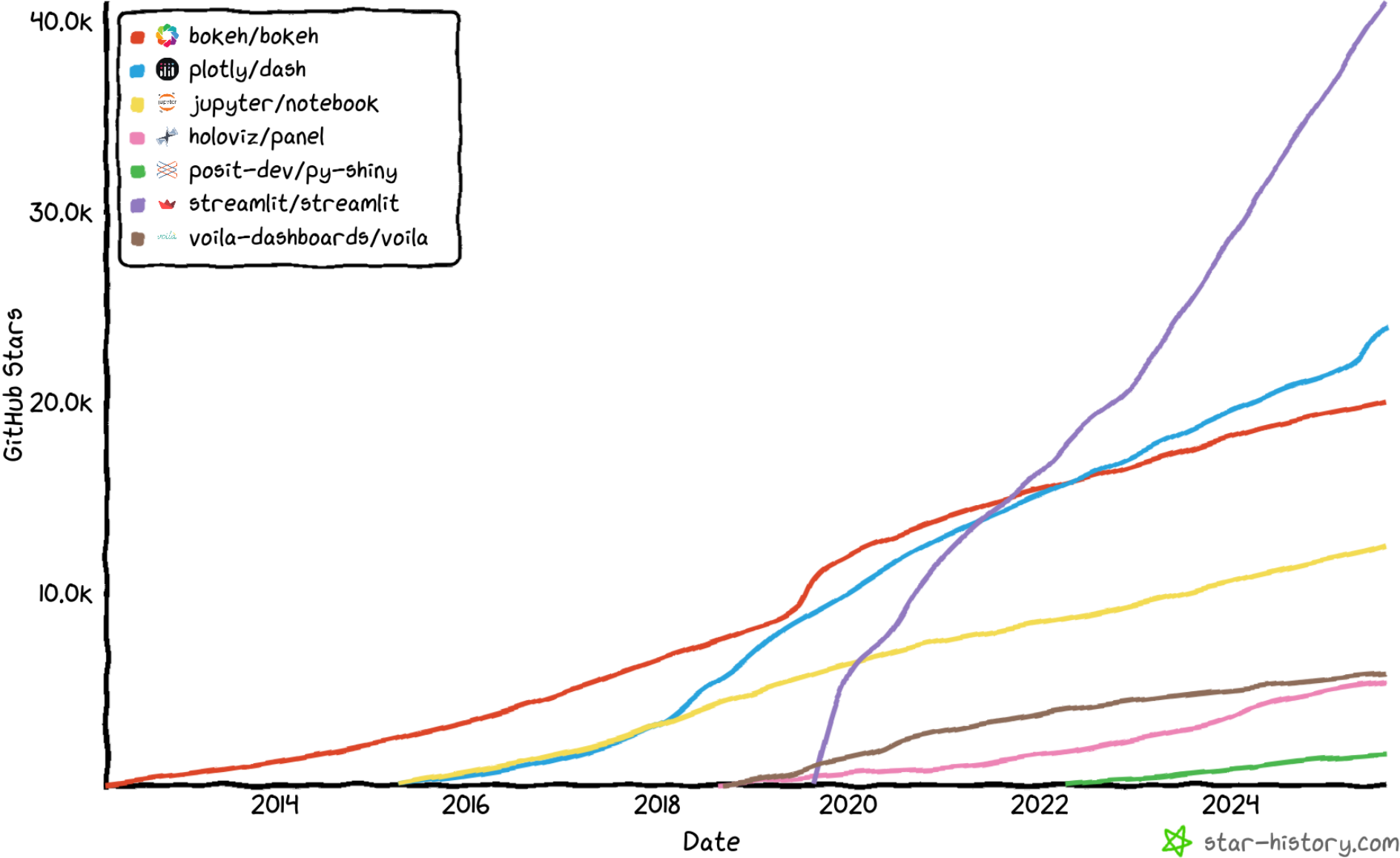
Quelques exemples: <https://awesome-panel.org/>

Frameworks web Python

Les plus connus pour la visualisation de données :

- [Bokeh](#) : la puissance au prix de l'exigence.
- [Dash](#) : solide et mature mais orienté Plotly.
- [Jupyter Notebook](#) : les notebooks (mais pas que ça).
- [Panel](#) : choix pour la suite, jeunesse et souplesse 🤸
- [Shiny for Python](#) : le framework venu du monde de R.
- [Streamlit](#) : simple et efficace, un autre bon choix 💪
- [Voilà](#) : la simplicité et ses limites 🙋

Star History



Panel

[Panel](#) est un module Python dédié au développement d'applications web telles que les tableaux de bord interactifs



Ce module faisait initialement partie de [Anaconda](#) à sa création en 2018 avant de rejoindre l'écosystème [Holoviz](#).

Panel est un **logiciel libre** distribué sous licence BSD.

Un avantage de Panel est de permettre le développement d'applications web via des *scripts* ou des *notebooks*, ce qui facilite le passage de l'expérimentation à la production 😊

Première application

```
1 import panel as pn
2
3 pn.extension()
4
5 pn.panel("Premiers pas avec Panel 🤖").servable()
```

- `panel` : module importé et son alias classique `pn`,
- `pn.extension` : permet de charger des dépendances *JavaScript* pour Panel,
- `pn.panel` : un élément affichable (dans un instant),
- `.servable()` : transmet l'élément au *serveur* d'application pour être **affiché dans l'application web**.

Démarrer l'application

Panel permet de placer le code précédent dans un script Python `mon_app.py` ou dans une cellule de code d'un notebook `mon_app.ipynb`.

```
1 # Pour un script
2 panel serve --autoreload --show mon_app.py
3
4 # Pour un notebook
5 panel serve --autoreload --show mon_app.ipynb
```

L'application est disponible (jusqu'à la fermeture avec `Ctrl+C`) dans un navigateur via le lien donné par le terminal.

What else ? 

Commande *panel serve*

Quelques options de la commande `panel serve` :

- `--show` Ouvre l'app dans un navigateur
- `--autoreload` Recharge l'app quand le code est modifié
- `--dev` Recharge l'app si un fichier est modifié (e.g. csv, ...)
- `--port` Configure le port d'écoute
- `--address` Configure l'adresse d'écoute
- `--log-level` Granularité des messages
- `--ico-path` Configure la *favicon*

Panel avec Onyxia

Panel et ses dépendances dont `watchfiles` (nécessaire pour `--autoreload`) peuvent être installés avec `pip` :

```
1 pip install panel jupyter_bokeh nbconvert watchfiles
```

Un premier essai ouvre une page vide 🤔 avec le message :

```
Refusing websocket connection from Origin 'https://user-...-  
ensae-745923-0.lab.groupe-genes.fr'; use --allow-  
websocket-origin=user-...-ensae-745923-0.lab.groupe-genes.fr or set  
BOKEH_ALLOW_WS_ORIGIN=user-...-ensae-745923-0.lab.groupe-genes.fr to permit  
this; currently we allow origins {'localhost:5006'}
```

Par sécurité, la connexion sans configuration supplémentaire est limitée à `localhost:5006` mais cela n'a pas de sens dans un environnement tel que Onyxia.

Il y a plusieurs façons de corriger cela en autorisant les connexions depuis le proxy utilisé par l'instance de VSCode comme indiqué dans le message d'erreur.

Le plus simple est d'ajouter `--allow-websocket-origin` (cf. le message d'erreur) à la commande `panel serve` ou d'utiliser la ligne de commande :

```
1 --allow-websocket-origin=$(echo $VSCODE_PROXY_URI | cut -d '/' -f 3)
```

Afficher des éléments

Pour construire une application web, il faut afficher des éléments (texte, image, graphique, ...).

La fonction `pn.panel` permet cela très facilement pour une grande variété d'éléments ainsi que des objets Python.

Nous avons déjà vu l'exemple d'un texte :

```
1 pn.panel("Premiers pas avec Panel 🤖").servable()
```



Remarque

Dans les exemples suivants, la méthode `servable` doit être invoquée pour obtenir l'affichage de l'élément dans l'application. Elle n'est pas nécessaire pour simplement tester l'élément dans le notebook.

Il est intéressant de remarquer que `pn.panel` génère un composant adapté à l'élément à afficher :

```
1 texte = pn.panel("Premiers pas avec Panel 🤖")
2 print(texte)
```

```
Markdown(str)
```

Pour du texte, il s'agit d'un `Markdown`. Cela signifie que cette syntaxe peut être utilisée pour mettre en forme le texte :

```
1 pn.panel("""
2 # Mon application
3
4 Les étudiants du [Cepe](https://www.lecepe.fr/) sont :
5
6 - *motivés*,
7 - **enthousiastes**,
8 - <span style="color:red">formidables</span>,
9 - ...
10 """).servable()
```

Quelques éléments affichables :

- Image

```
1 pn.panel("data/gandalf.jpg", width=256).servable()
```

- Média

```
1 pn.panel("data/gandalf.mp3").servable()
```

- Objet Python

```
1 notes = {  
2     "Nom": ["Teddy", "Jenny", "Cindy", "Franky", "Jimmy"],  
3     "Note": [12, 18, 14, 12, 16],  
4 }  
5  
6 pn.panel(notes).servable()
```

- DataFrame Pandas

```
1 import pandas as pd
2
3 df = pd.DataFrame(notes)
4
5 pn.panel(df).servable()
```

- Graphique Matplotlib

```
1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots()
4 ax.bar(df.Nom, df.Note)
5 ax.grid(linestyle=":")
6
7 plt.close(fig) # Important pour éviter les fuites de mémoire
8
9 pn.panel(fig, width=512).servable()
```

- Graphique Plotly

```
1 import plotly.express as px
2
3 pn.extension("plotly") # Nécessaire pour Plotly
4
5 fig_plotly = px.line(df, x="Nom", y="Note")
6 fig_plotly.update_traces(mode="lines+markers")
7 fig_plotly.layout.autosize = True
8
9 pn.panel(fig_plotly).servable()
```

- Et bien d'autres éléments : voir la section *Panes* de la [Component Gallery](#) de Panel 😎

Performance

La fonction `pn.panel` est pratique car elle génère un composant adapté à l'élément. En pratique, il est néanmoins plus performant d'utiliser explicitement le *Pane* voulu.

Layout

Les éléments sont affichés en colonne par défaut. Il va falloir faire mieux pour une application web digne de ce nom !

La section *Layouts* de la [Component Gallery](#) liste toutes les possibilités de mise en forme proposées par Panel.

En pratique, nous utilisons surtout des combinaisons de :

- `pn.Column` pour de éléments en colonne,
- `pn.Row` pour des éléments en ligne.

La méthode `servable` n'est ainsi invoquée que sur le conteneur `Column` global.

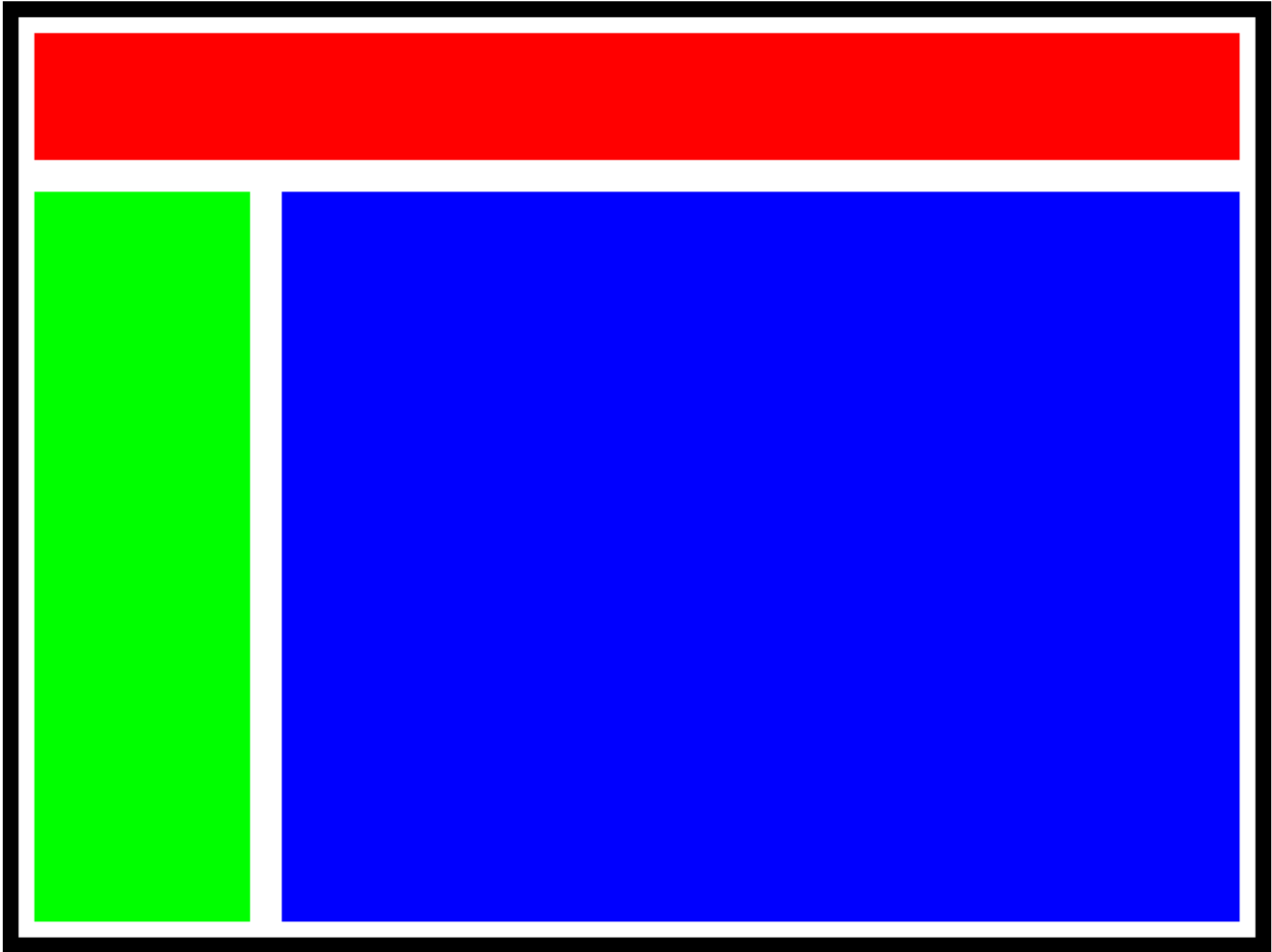
```
1 pn.Column(  
2     pn.Row(  
3         pn.pane.JPG("data/gandalf.jpg", width=256),  
4         pn.Column(  
5             pn.pane.Audio("data/gandalf.mp3"),  
6             pn.Row(pn.pane.JSON(notes), pn.pane.DataFrame(df)),  
7         ),  
8     ),  
9     pn.Row(  
10        pn.pane.Matplotlib(  
11            fig, height=400, sizing_mode="stretch_width"  
12        ),  
13        pn.pane.Plotly(  
14            fig_plotly, height=400, sizing_mode="stretch_width"  
15        ),  
16    ),  
17 ).servable()
```

Templates

Au-delà de la simple mise en forme (*layout*), Panel propose différents modèles (*templates*) pour organiser une application web en grille, slides, ... (cf. *Templates* de la [Component Gallery](#))

Pour la suite, nous ne retiendrons que les modèles *sidebar* tels que `FastListTemplate` avec les zones :

- `header` En-tête de l'application
- `sidebar` Bloc latéral (rétractable)
- `main` Bloc principal
- `modal` Bloc optionnel pouvant être masqué/affiché



Un exemple avec une zone d'en-tête turquoise

```
1  template = pn.template.FastListTemplate(  
2      title="Ma belle application",  
3      header_background="teal",  
4      sidebar=[  
5          pn.pane.JPG("data/gandalf.jpg", sizing_mode="stretch_width")  
6      ],  
7  )  
8  
9  template.main.append(  
10     pn.Row(  
11         pn.pane.DataFrame(df),  
12         pn.pane.JSON(notes),  
13     ),  
14 )  
15  
16 template.servable()
```

Widgets

Il est temps de rendre nos applications **réactives** : l'utilisateur doit pouvoir faire des choix et l'application y réagir.

Un élément permettant l'action de l'utilisateur vers l'application est un **input** par opposition à un **output** qui ne fait que retourner de l'information (image, texte, ...).

Panel propose de nombreux éléments pouvant servir d'input (et il est possible d'en créer des nouveaux) dont les **widgets** :

Widgets dans la Component Gallery

Les widgets possèdent généralement un attribut **value** qui permet d'accéder à la valeur donnée par l'utilisateur.

Quelques exemples :

- **Checkbox** Case à cocher (**True/False**)

```
1 pn.widgets.Checkbox(name="Ma case")
```

- **TextInput** Saisie textuelle (**str**)

```
1 pn.widgets.TextInput(name="Nom", placeholder="Luke Skywalker")
```

- **IntSlider** Choix d'entier (**int**)

```
1 pn.widgets.IntSlider(name="Note", start=1, end=10, value=5)
```

- ...

Réactivité

Pour que les widgets soient utiles, il faut que l'application réagisse lorsqu'ils sont modifiés.

La méthode *classique* consiste à utiliser `pn.bind` pour surveiller (*watch*) l'état d'un widget et appeler une fonction (*callback*) quand il change de valeur.

```
1 def print_note(note):  
2     return f"Ma note est {note}/10"  
3  
4 ma_note = pn.widgets.IntSlider(name="Note", start=1, end=10, value=5)  
5  
6 note = pn.bind(print_note, note=ma_note)  
7  
8 pn.Column(ma_note, note).servable()
```

L'option `watch` est utile si la fonction liée ne retourne rien.

Bien entendu, la fonction `pn.bind` peut être utilisée avec plusieurs widgets :

```
1 def print_note(note, couleur):
2     return (
3         "Ma note est "
4         + f"<span style='color:{couleur}'>{note}/10</span>"
5     )
6
7 ma_note = pn.widgets.IntSlider(name="Note", start=1, end=10, value=5)
8 ma_couleur = pn.widgets.Select(
9     name="Couleur", options=["red", "blue"]
10 )
11
12 note = pn.bind(print_note, note=ma_note, couleur=ma_couleur)
13
14 pn.Column(ma_note, ma_couleur, pn.pane.Markdown(note)).servable()
```

Notez l'usage de `Markdown` pour la performance en évitant que Panel doive “deviner” le type de l'élément.

Chargement

Pour fonctionner, une application web a généralement besoin de charger des ressources extérieures :

- modules Python,
- jeux de données,
- ...

❗ Où ces chargements doivent-ils avoir lieu ?

Il est **important** de se poser cette question pour ne pas alourdir inutilement votre application et détériorer l'expérience utilisateur !

Prenons l'exemple suivant :

```
1 import pandas as pd
2 import panel as pn
3
4 pn.extension()
5
6 def get_data(n):
7     return pd.read_csv("data/diamonds.csv").head(n)
8
9 n = pn.widgets.IntSlider(name="Lignes", start=1, end=25, value=5)
10 df = pn.bind(get_data, n=n)
11 pn.Column(n, df).servable()
```

Le code est exécuté en partie ...

Prenons l'exemple suivant :

```
1 import pandas as pd
2 import panel as pn
3
4 pn.extension()
5
6 def get_data(n):
7     return pd.read_csv("data/diamonds.csv").head(n)
8
9 n = pn.widgets.IntSlider(name="Lignes", start=1, end=25, value=5)
10 df = pn.bind(get_data, n=n)
11 pn.Column(n, df).servable()
```

Le code est exécuté en partie ...

- ... au démarrage de l'application,

Prenons l'exemple suivant :

```
1 import pandas as pd
2 import panel as pn
3
4 pn.extension()
5
6 def get_data(n):
7     return pd.read_csv("data/diamonds.csv").head(n)
8
9 n = pn.widgets.IntSlider(name="Lignes", start=1, end=25, value=5)
10 df = pn.bind(get_data, n=n)
11 pn.Column(n, df).servable()
```

Le code est exécuté en partie ...

- ... au démarrage de l'application,
- ... quand un utilisateur se connecte,

Prenons l'exemple suivant :

```
1 import pandas as pd
2 import panel as pn
3
4 pn.extension()
5
6 def get_data(n):
7     return pd.read_csv("data/diamonds.csv").head(n)
8
9 n = pn.widgets.IntSlider(name="Lignes", start=1, end=25, value=5)
10 df = pn.bind(get_data, n=n)
11 pn.Column(n, df).servable()
```

Le code est exécuté en partie ...

- ... au démarrage de l'application,
- ... quand un utilisateur se connecte,
- ... quand un widget est modifié.

Cache

Lorsqu'une fonction retourne un résultat *déterministe* et indépendant de l'utilisateur, il peut être mis en **cache** pour améliorer les performances de l'application (surtout si ce résultat demande un temps de calcul non négligeable).

Un résultat pré-calculé pour des paramètres donnés sera alors retourné instantanément sans que la fonction soit à nouveau évaluée.

Pour utiliser le cache, la fonction doit être **décorée** avec `pn.cache`.

Améliorons le code précédent en utilisant le cache pour la fonction `get_data` :

```
1 import pandas as pd
2 import panel as pn
3
4 pn.extension()
5
6 @pn.cache
7 def get_data(n):
8     return pd.read_csv("data/diamonds.csv").head(n)
9
10 n = pn.widgets.IntSlider(
11     name="Nombre de lignes", start=1, end=25, value=5
12 )
13 df = pn.bind(get_data, n=n)
14 pn.Column(n, df).servable()
```


Exemple complet

Cette application web sera améliorée dans la partie pratique et permet de visualiser la fréquence des éruptions du geyser **Old Faithful** en fonction du temps d'attente.

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import panel as pn
4
5 pn.extension()
6
7 df = pd.read_csv("data/faithful.csv")
8
9 bins = pn.widgets.IntSlider(
10     name="Nombre de blocs", start=1, end=50, value=30
11 )
```

```
1 @pn.cache
2 def faithful_histogram(bins):
3     fig, ax = plt.subplots()
4     ax.hist(df.waiting, bins=bins)
5     ax.set_xlabel("Temps avant la prochaine éruption (min)")
6     ax.set_ylabel("Fréquence")
7     plt.close(fig)
8     return fig
9
10 histogram = pn.pane.Matplotlib(pn.bind(faithful_histogram, bins=bins))
11
12 pn.template.FastListTemplate(
13     title="Old Faithful Geyser",
14     sidebar=[bins],
15     main=[histogram],
16 ).servable()
```

Il n'y a plus qu'à démarrer cette application avec `panel serve`



Aller plus loin

Vous êtes maintenant prêts à créer vos applications web ! 🎉

Pour aller plus loin, vous pouvez regarder les exemples de la [App Gallery](#) et la documentation sur des thèmes complémentaires :

- Alternatives à `pn.bind` : `pn.depends`, `on_click`, ...
- Réactivité avec `pn.rx`
- Personnaliser les visuels avec l'option `styles`
- Lien avec les autres modules [Holoviz](#)
- Création de widgets complexes avec [Param](#)

À vous de jouer !